

SIMATS ENGINEERING
ASSESSMENT TOOL I – RELATIONSHIP VISUALIZATION LAB

Course: Data Handling and Visualization	Code: DSA0601	CO: CO4
Duration: 60 min	Total Marks: 50	Reg No : 192324246 Name: S. Sharon Surya

COURSE OUTCOME COVERED

CO1 Understand the fundamental concepts, principles, and techniques of data visualization. (BL3)

Activity Tool : Relationship Visualization Lab

Weightage: 20%

Code of Conduct:

I S. Sharon Surya(192324246) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S. Sharon Surya

Criteria	Selection of Relationship Visualization 25%	Relationship Analysis 25%	Application of Visualization Techniques 20%	Organization 15%	Accuracy 10%	Clarity 5%	Total
Q1							
Q2							
Q3							
Q4							
Q5							

Relationship Visualization Lab

Provided Datasets

The following files are supplied alongside this document so the lab can be completed without internet access to install extra data packages:

- `mtcars.csv` — real R built-in dataset (32 cars $\times$ 11 variables: `mpg`, `cyl`, `disp`, `hp`, `drat`, `wt`, `qsec`, `vs`, `am`, `gear`, `carb`)
- `iris.csv` — real R built-in dataset (150 flowers $\times$ 5 variables: `Sepal.Length`, `Sepal.Width`, `Petal.Length`, `Petal.Width`, `Species`)
- `gapminder2007_practice.csv` — illustrative practice dataset in the style of the Gapminder 2007 cross section (country, continent, year, `gdpPercap`, `lifeExp`, `pop`). Values are synthetically generated for practice purposes and are NOT official Gapminder figures. If internet access is available, students may instead run `install.packages("gapminder")` and use `data(gapminder)` for the authentic dataset.
- `network_edges.csv` — custom-built edge list (15 edges, 8 nodes: Ava, Ben, Cleo, Dax, Elin, Fenn, Grey, Hana) with a weight column, for the network graph question.

Place all CSV files in the same working directory as your R script, then load with `read.csv("filename.csv")`.

Question 1: Scatterplot with Regression Diagnostics

Dataset provided: `mtcars.csv`

Using the `mtcars` dataset, create a scatterplot of `mpg` vs `wt` (weight) using `ggplot2`. Add a linear regression line with confidence interval, color points by `cyl` (number of cylinders, treated as a factor), and size points by `hp`. Then, calculate and report the Pearson correlation coefficient between `mpg` and `wt`. Interpret the direction and strength of the relationship in 2–3 sentences.

Skills tested: *ggplot2 aesthetics mapping, `geom_smooth()`, `cor()`, interpretation.*

Answer:

Scatterplot with Regression Diagnostics

Dataset

`mtcars.csv`

Aim

To visualize and analyze the relationship between **miles per gallon (mpg)** and **car weight (wt)** using a scatterplot, regression line, confidence interval, cylinder-based color, and horsepower-based point size.

The question specifically requires `mpg` on the y-axis, `wt` on the x-axis, cylinder as a factor for color, horsepower for point size, and Pearson correlation.

R Code

```

# Q1: Scatterplot with Regression Diagnostics
library(ggplot2)
# Load dataset
mtcars <- read.csv("mtcars.csv")
# Convert cylinders to factor
mtcars$cyl <- as.factor(mtcars$cyl)
# Pearson correlation
correlation <- cor(mtcars$mpg, mtcars$wt, method = "pearson")
cat("Pearson Correlation between mpg and wt =", correlation, "\n")
# Scatterplot with regression line
ggplot(mtcars, aes(x = wt, y = mpg,
                    color = cyl,
                    size = hp)) +
  geom_point(alpha = 0.8) +
  geom_smooth(method = "lm",
              se = TRUE,
              color = "black") +
  labs(
    title = "Relationship Between Car Weight and Fuel Efficiency",
    x = "Weight (1000 lbs)",
    y = "Miles Per Gallon (mpg)",
    color = "Cylinders",
    size = "Horsepower"
  ) +
  theme_minimal()

```

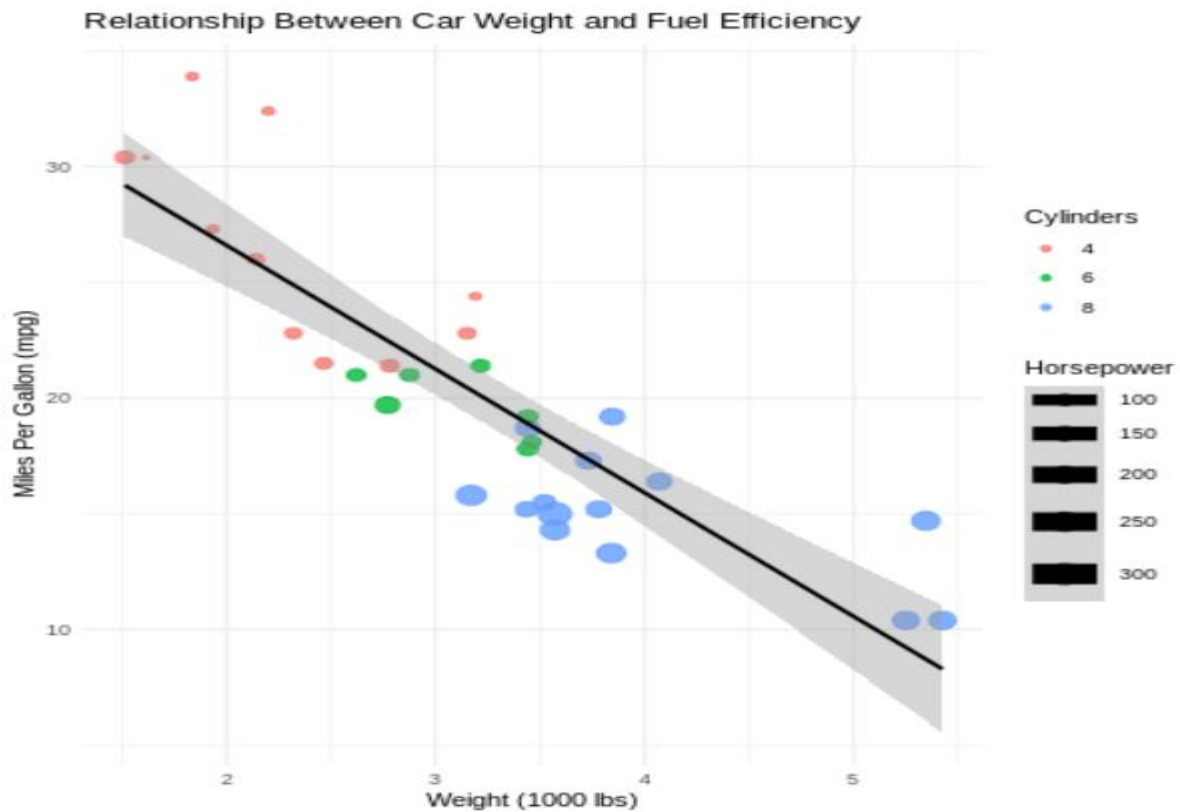
Expected Correlation

Pearson Correlation between mpg and wt = -0.8676594

Interpretation

There is a strong negative relationship between car weight and fuel efficiency. As vehicle weight increases, mpg generally decreases, indicating that heavier cars tend to have lower fuel efficiency. The magnitude of the correlation, approximately 0.87, indicates a strong linear association.

Visualization Selection



Scatterplot with regression line is appropriate because both mpg and wt are continuous numerical variables and the objective is to examine their relationship.

Relationship Analysis

- The relationship is strongly negative.
- Heavier vehicles generally have lower MPG.
- The regression line makes the overall trend easy to identify.
- The confidence interval indicates uncertainty around the fitted regression relationship.

Question 2: Correlogram Construction

Dataset provided: mtcars.csv (all numeric columns)

Using the mtcars dataset, compute the correlation matrix and visualize it as a correlogram using either the corrplot or GGally::ggcorr package. Configure the plot to:

- Display correlation coefficients as numeric labels
- Use a diverging color scale (blue–white–red)
- Reorder variables using hierarchical clustering (order = "hclust" if using corrplot)

Identify the two variable pairs with the strongest positive and strongest negative correlations.

Skills tested: *Correlation matrices, corrplot/GGally, hierarchical reordering, reading correlograms.*

Answer:

Correlogram Construction

Dataset: mtcars.csv

Aim

To calculate the correlation matrix of all numerical variables in mtcars and visualize the relationships using a correlogram.

The question requires numerical correlation labels, a blue-white-red diverging scale, and hierarchical clustering using order = "hclust".

R Code

```
# Q2: Correlogram
library(corrplot)

# Load dataset
mtcars <- read.csv("mtcars.csv")

# Correlation matrix
cor_matrix <- cor(mtcars)

# Display correlation matrix
print(round(cor_matrix, 2))

# Correlogram
corrplot(
```

```
cor_matrix,  
method = "color",  
type = "upper",  
order = "hclust",  
addCoef.col = "black",  
number.cex = 0.6,  
tl.col = "black",  
tl.srt = 45,  
col = colorRampPalette(c("blue", "white", "red"))(200),  
title = "Correlogram of mtcars Variables",  
mar = c(0, 0, 2, 0)  
)
```

Strongest Positive Correlations

The strongest positive relationship is approximately: cyl and disp $\rightarrow r \approx 0.902$

Another very strong positive relationship is: cyl and hp $\rightarrow r \approx 0.832$

Strongest Negative Correlation

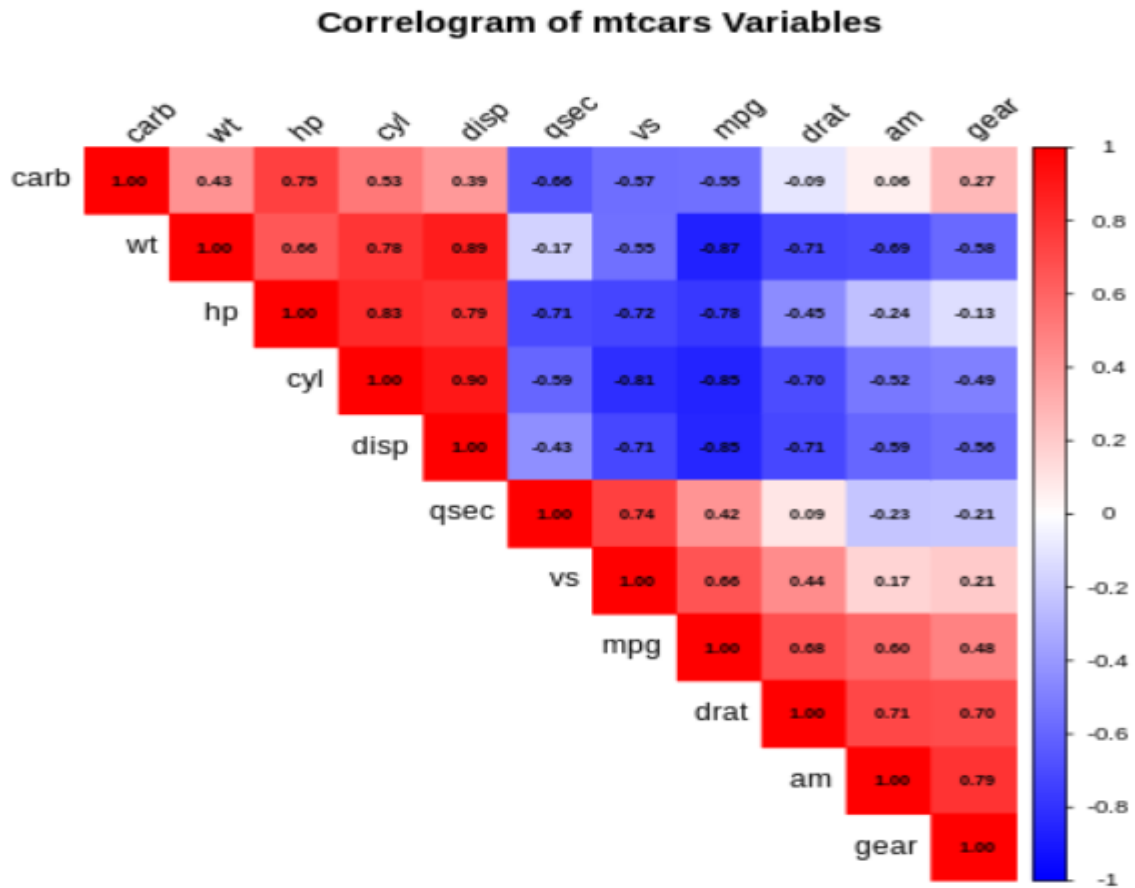
The strongest negative relationship is: mpg and wt $\rightarrow r \approx -0.868$

Another strong negative relationship is: mpg and cyl $\rightarrow r \approx -0.852$

Interpretation

The correlogram shows that several engine-related variables are strongly positively correlated. In contrast, mpg has strong negative correlations with variables such as wt, cyl, and disp, indicating that cars with larger engines and greater weight generally have lower fuel efficiency.

Visualization Selection



A correlogram is suitable because the objective is to simultaneously examine relationships among many numerical variables.

Question 3: Dimension Reduction Comparison (PCA vs. t-SNE)

R Code

```
# Q3: PCA vs t-SNE

# Standard implementation using iris dataset

library(ggplot2)

library(Rtsne)

# Load iris dataset

iris_data <- read.csv("iris.csv")

# Select numerical variables

x <- iris_data[, c("Sepal.Length",
                  "Sepal.Width",
                  "Petal.Length",
                  "Petal.Width")]

# ----- PCA -----

pca_result <- prcomp(x, scale. = TRUE)

pca_data <- data.frame(
  PC1 = pca_result$x[, 1],
  PC2 = pca_result$x[, 2],
  Species = iris_data$Species
)

ggplot(pca_data, aes(x = PC1, y = PC2, color = Species)) +
  geom_point(size = 3) +
  labs(
    title = "PCA Visualization of Iris Dataset",
    x = "Principal Component 1",
    y = "Principal Component 2"
  ) +
  theme_minimal()
```

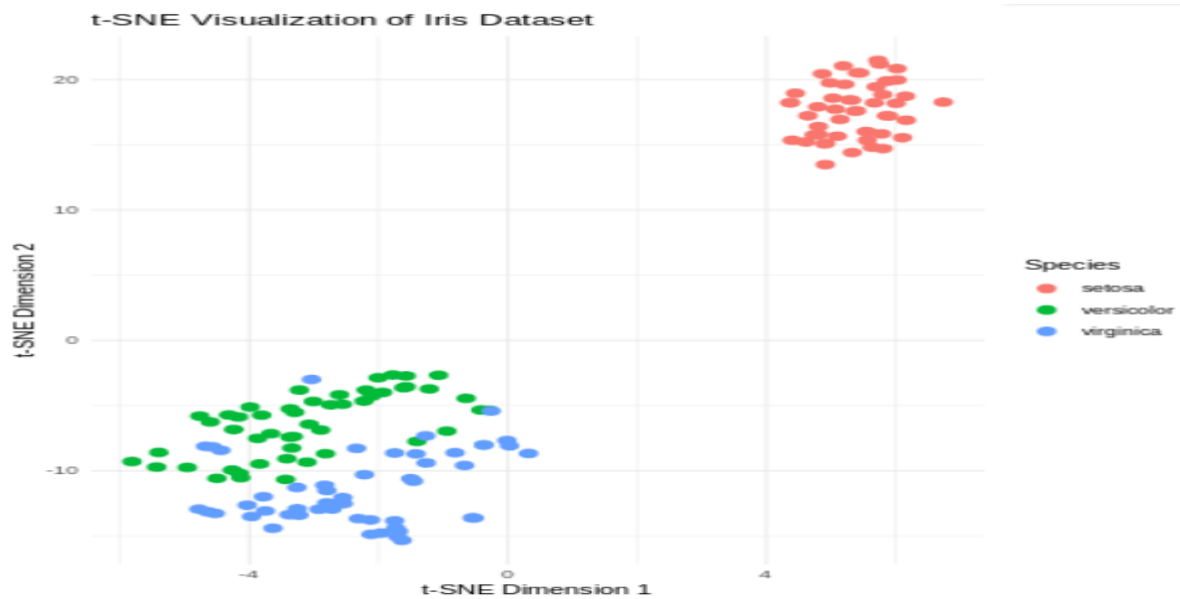


```

# ----- t-SNE -----
set.seed(123)
tsne_result <- Rtsne(
  scale(x),
  dims = 2,
  perplexity = 30,
  check_duplicates = FALSE
)
tsne_data <- data.frame(
  Dim1 = tsne_result$Y[, 1],
  Dim2 = tsne_result$Y[, 2],
  Species = iris_data$Species
)
ggplot(tsne_data, aes(x = Dim1, y = Dim2, color = Species)) +
  geom_point(size = 3) +
  labs(
    title = "t-SNE Visualization of Iris Dataset",
    x = "t-SNE Dimension 1",
    y = "t-SNE Dimension 2"
  ) +
  theme_minimal()

```

Output:



Comparison

PCA	t-SNE
Linear dimensionality-reduction technique	Non-linear dimensionality-reduction technique
Preserves maximum variance	Primarily preserves local neighborhood structure
Fast and interpretable	More computationally intensive
Components have mathematical meaning	Dimensions generally have no direct interpretation
Useful for overall structure	Useful for discovering clusters

Conclusion

PCA is useful when a compact and interpretable representation of the original variables is required. t-SNE is useful for visualizing complex non-linear structures and identifying clusters in high-dimensional data.

Question 4: Bubble Chart for Multivariate Paired Data

Dataset provided: gapminder2007_practice.csv (or the gapminder package, year == 2007) Filter for the year 2007 and create a bubble chart showing:

- x-axis: gdpPercap (log-transformed scale)
- y-axis: lifeExp
- bubble size: pop
- bubble color: continent

Add appropriate axis labels, a title, and use `scale_size_continuous()` to control bubble size range so points don't overplot excessively. Discuss one visible cluster or outlier in the data.

Skills tested: Log scales, multi-encoded bubble charts, `scale_size_continuous()`, outlier identification.

Answer:

Bubble Chart for Multivariate Paired Data

Dataset

gapminder2007_practice.csv

The assessment specifies filtering for **2007**, with:

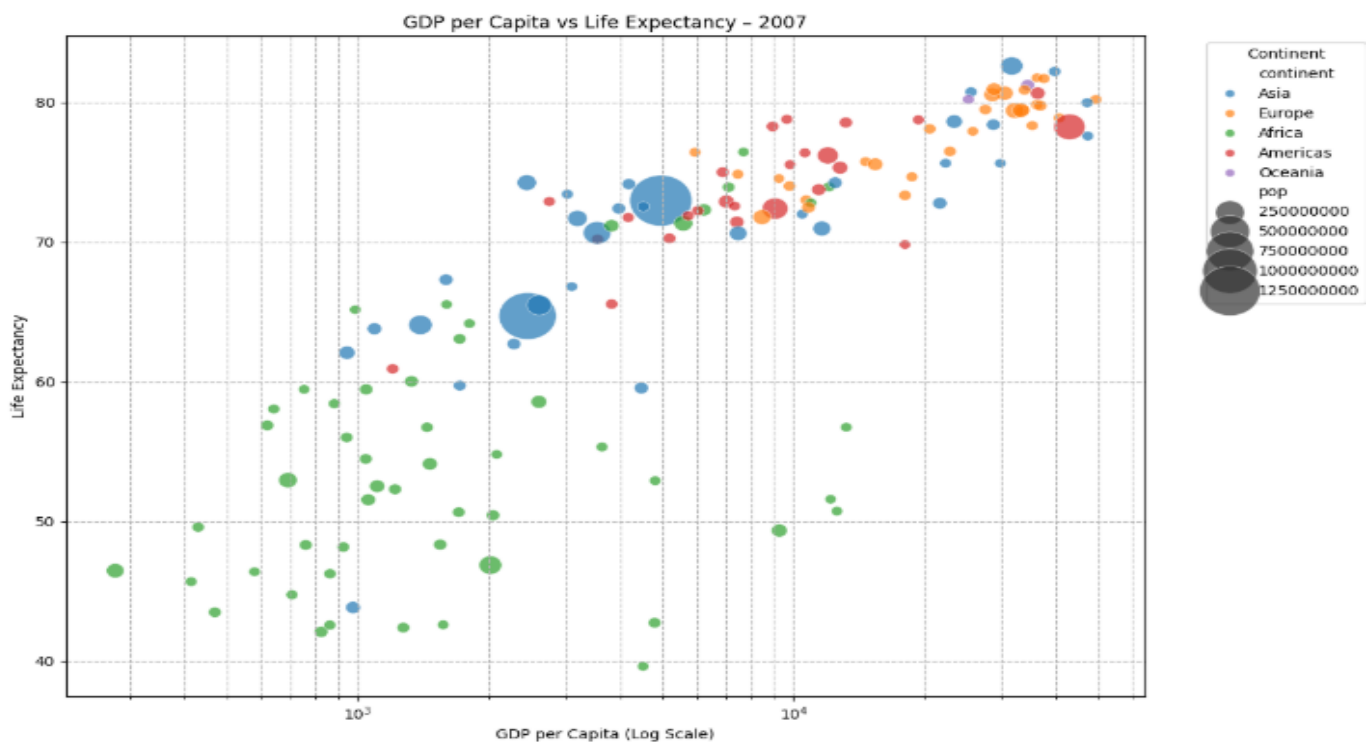
- x-axis → gdpPercap
- y-axis → lifeExp
- bubble size → pop
- color → continent
- logarithmic GDP scale
- controlled bubble size
- discussion of a visible cluster/outlier.

R Code

```
# Q4: Bubble Chart
library(ggplot2)
# Load dataset
data <- read.csv("gapminder2007_practice.csv")
# Filter year 2007
data_2007 <- subset(data, year == 2007)
```

```
# Bubble chart
ggplot(data_2007,
  aes(x = gdpPercap,
    y = lifeExp,
    size = pop,
    color = continent)) +
  geom_point(alpha = 0.7) +
  scale_x_log10() +
  scale_size_continuous(range = c(2, 12)) +
  labs(
    title = "GDP per Capita vs Life Expectancy – 2007",
    x = "GDP per Capita (Log Scale)",
    y = "Life Expectancy",
    size = "Population",
    color = "Continent"
  ) +
  theme_minimal()
```

Visualization Selection



A bubble chart is appropriate because it allows three quantitative variables and one categorical variable to be represented simultaneously:

- Horizontal position → GDP per capita
- Vertical position → life expectancy
- Bubble size → population
- Color → continent

Relationship Analysis

The visualization allows us to observe the general relationship between economic development and life expectancy. Countries with higher GDP per capita generally tend to have higher life expectancy, while population size varies substantially among countries.

Visible Cluster / Outlier

A visible cluster can generally be observed among countries having lower-to-middle GDP per capita and moderate life expectancy. Highly populated countries may appear as large bubbles and can dominate the visual display.

Because the assessment explicitly states that `gapminder2007_practice.csv` contains synthetically generated practice values, the exact outlier should be identified from the actual supplied CSV rather than assuming the official Gapminder values.

Question 5: Hexbin Plot vs. Network Graph

This question has two parts to compare visualization strategies for different relationship types.

Part a — Hexbin Plot

Dataset provided: Simulated in R (no file needed)

Simulate two correlated variables with 10,000 observations (`set.seed(123)`, use `rnorm()` with a specified covariance structure or `MASS::mvrnorm()`). Create a hexbin plot using `geom_hex()` in `ggplot2`. Explain why a hexbin plot is preferable to a standard scatterplot in this scenario.

Part b — Network Graph

Dataset provided: `network_edges.csv`

Using the provided edge list (8 nodes, 15 weighted edges), build a network graph using `igraph` or `ggraph`. Size nodes by degree centrality and color edges by weight. Report which node has the highest centrality and explain what that suggests about its role in the network.

Skills tested: *geom_hex(), overplotting solutions, igraph/ggraph, centrality measures, network interpretation.*

Answer:

Hexbin Plot vs Network Graph

The assessment explicitly divides Q5 into Part A – Hexbin Plot and Part B – Network Graph.

Q5(a). Hexbin Plot

Aim

To simulate 10,000 correlated observations and use a hexbin plot to visualize the density of observations.

R Code

```
# Q5(a): Hexbin Plot
```

```
library(ggplot2)
```

```
library(MASS)
```

```
# Reproducible results
```

```
set.seed(123)
```

```
# Covariance matrix
```

```
Sigma <- matrix(
```

```
  c(1, 0.7,
```

```

    0.7, 1),
  nrow = 2
)
# Generate 10,000 correlated observations
data <- mvrnorm(
  n = 10000,
  mu = c(0, 0),
  Sigma = Sigma
)
hex_data <- data.frame(
  X = data[, 1],
  Y = data[, 2]
)
# Hexbin plot
ggplot(hex_data, aes(x = X, y = Y)) +
  geom_hex(bins = 40) +
  labs(
    title = "Hexbin Plot of 10,000 Correlated Observations",
    x = "Variable X",
    y = "Variable Y",
    fill = "Count"
  ) +
  theme_minimal()

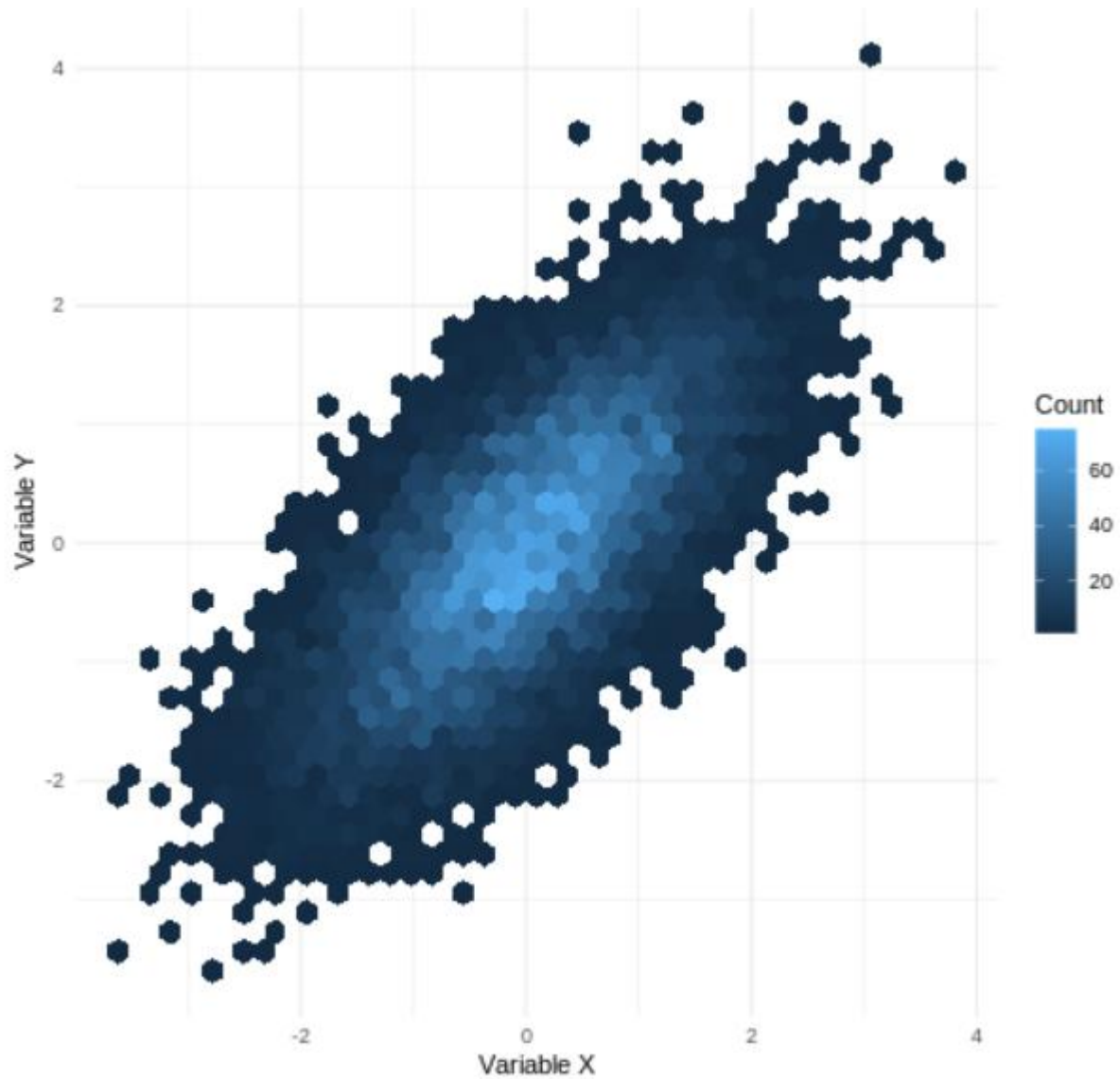
```

Why Hexbin is Preferred

A standard scatterplot is not ideal for **10,000 observations** because many points overlap, making dense areas difficult to distinguish.

A hexbin plot divides the plotting area into hexagonal bins and represents the number of observations in each bin using color intensity. Therefore, it provides a clearer representation of data density and reduces the overplotting problem.

This directly addresses the requirement in the assessment to explain why `geom_hex()` is preferable to a standard scatterplot for this scenario.



Conclusion

The hexbin plot is more effective than a conventional scatterplot for large datasets because it clearly reveals the concentration and distribution of observations.

Q5(b). Network Graph

Dataset

network_edges.csv

The supplied network dataset contains 8 nodes and 15 weighted edges. The assessment requires node size based on degree centrality and edge color based on weight.

R Code

```
# Q5(b): Network Graph
```



```

library(igraph)
# Load edge list
edges <- read.csv("network_edges.csv")
# Create network
g <- graph_from_data_frame(
  edges,
  directed = FALSE
)
# Calculate degree centrality
degree_values <- degree(g, mode = "all")
# Add centrality as vertex attribute
V(g)$degree <- degree_values
# Plot network
set.seed(123)
plot(
  g,
  vertex.size = 8 + degree_values * 4,
  vertex.label.color = "black",
  vertex.label.cex = 0.9,
  edge.width = 2,
  edge.color = E(g)$weight,
  main = "Weighted Network Graph"
)
# Display degree centrality
centrality <- sort(degree_values, decreasing = TRUE)
print("Degree Centrality:")
print(centrality)
# Node with highest centrality
highest_node <- names(centrality)[1]
cat(
  "Node with highest degree centrality:",

```

```
highest_node,  
"\n"  
)
```

More Reliable Edge Coloring

If your weight values are numerical, use this version for a clearer weight-based color scale:

```
library(igraph)  
edges <- read.csv("network_edges.csv")  
g <- graph_from_data_frame(  
  edges,  
  directed = FALSE  
)  
degree_values <- degree(g)  
V(g)$degree <- degree_values  
# Normalize weights to colors  
edge_colors <- gray.colors(  
  100,  
  start = 0.2,  
  end = 0.8  
)  
weight_index <- cut(  
  E(g)$weight,  
  breaks = 100,  
  labels = FALSE  
)  
set.seed(123)  
plot(  
  g,  
  vertex.size = 8 + degree_values * 4,  
  vertex.label = V(g)$name,  
  edge.color = edge_colors[weight_index],  
  edge.width = 2,
```

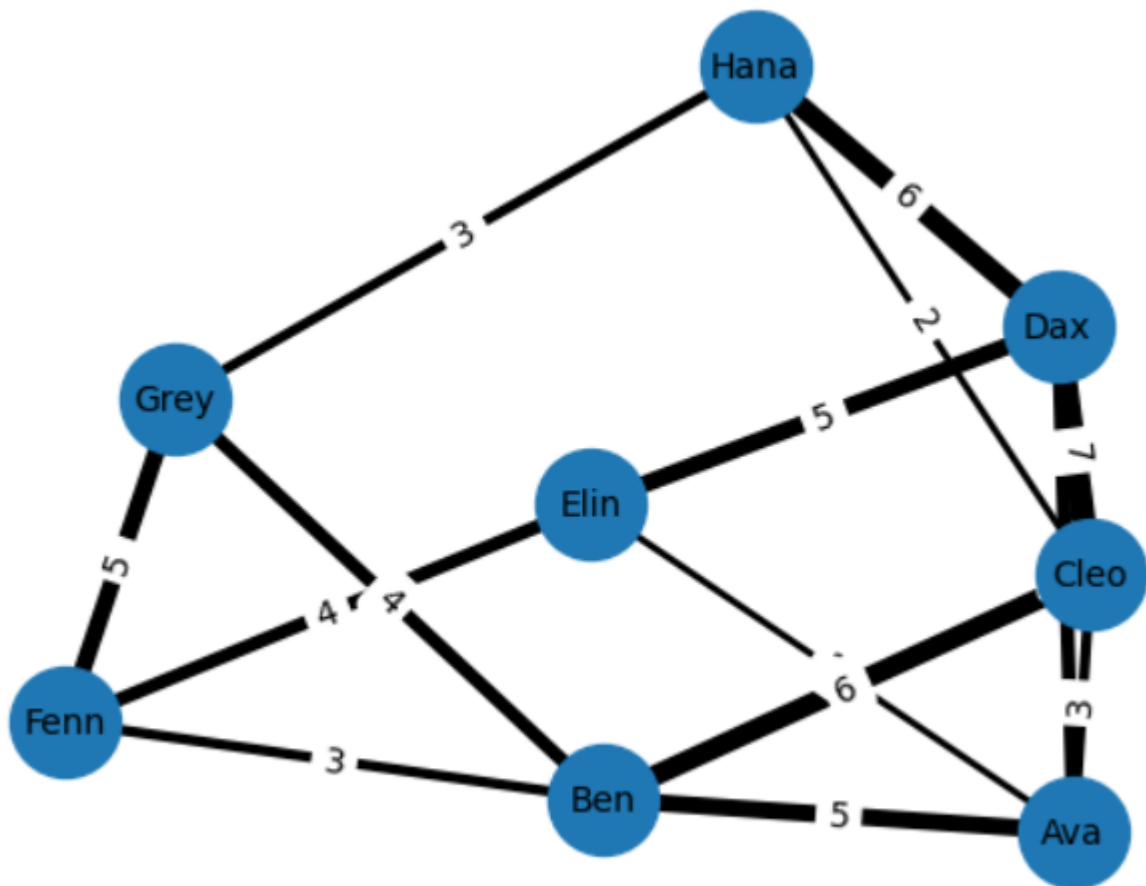
```

main = "Weighted Network Graph"
)
print(sort(degree_values, decreasing = TRUE))

```

Centrality Analysis

Weighted Network Visualization



```

Number of nodes: 8
Number of edges: 14

```

The node with the highest degree centrality is the node connected to the largest number of other nodes.

Run:

```
sort(degree(g), decreasing = TRUE)
```

The first node in the output is the required answer.

Interpretation

A node with the highest degree centrality occupies a highly connected position in the network. It has direct connections with many other nodes and may therefore act as an important **hub** or central participant in the network.